SDC HACKATHON 2026

Phase 2: Final Project & Prototype Report

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 1 November 2026

Instructions: Complete every field below and submit as a single PDF document named "SDC2026_TeamName_Final.pdf" together with your pitch deck and a working deployment link. Answers must comply with the word limits stated for each open-ended question. Section 3 requires concrete, verifiable evidence. Unsupported claims will not be credited by judges.

1. Team & Project Overview

Team Name	Yaponomics
Project / Product Name	Yapoz
Final Live Deployment URL	https://yapoz.vercel.app (use the demo buttons on the login page)
Codebase Repository URL	https://github.com/Ramilbey/yapoz
Name (Leader)	Abdulrahman Imaad Mohamed
Email & phone (Leader)	swe2409099@xmu.edu.my, +60173793811

2. Project Description

[Maximum 250 words]

Yapoz turns customer chats into orders that staff check and approve. It is built for micro and small shops that take orders on WhatsApp and Telegram, such as bakeries, salons and caterers.

Today the owner reads each message, looks up the price, checks stock, sends payment details and checks the receipt by eye. That is five manual steps per order, and mistakes cost sales.

In Yapoz, a customer message starts the work. An AI assistant answers from the owner's own menu, hours and policies, and can only state facts it reads from the database. Code prices the order, checks stock and holds the stock while the order waits. When the customer sends a payment receipt, the system compares amount, payee, date and reuse, and flags anything that does not match. The order stays pending until an owner or service user approves it. The backend blocks approval without the right role. A rejection needs a reason and puts the stock back. Every step is written to an activity log as it happens.

Each business is a workspace with its own catalogue and settings, so a second shop is a setup change, not a rebuild. We tested this with three fictional businesses: a pizzeria, a hair salon and a caterer.

Yapoz is designed as an order module for Orbyte. The Orbyte link is a documented proposal with example events, not a live connection. On 20 synthetic orders, manual steps fell from 5 to 1.10 per order.

3. Upgraded Product & Technical Execution

3.1 Final System Architecture

Provide a detailed description (and, if possible, an attached technical diagram) of your fully built system. Explain how your application integrates GenAI APIs, database structures, and backend/frontend frameworks into one unified solution.

Yapoz is one Next.js 16 (TypeScript) app on Vercel. It serves the staff dashboard and all webhook and API routes. Data is in PostgreSQL on Supabase, accessed with Drizzle ORM. Every row belongs to a business workspace. This is how one order moves through the system:

1. Customer message. It arrives from the WhatsApp Cloud API, the Telegram Bot API or the website simulator. The webhook signature is checked (HMAC for Meta, secret token for Telegram).

2. Queue. The message is saved as a job in PostgreSQL. Failed jobs retry, so a crash does not lose it.

3. Context. The pipeline loads the business's catalogue, hours, policies and FAQs (src/agent/). It is the same for every channel.

4. AI. The model replies and calls tools that read the database. It cannot set a price or invent a fact.

5. Rules. Code prices the order, checks stock and holds it, then saves a pending order (src/lib/orders.ts, stock.ts).

6. Receipt. A vision model reads the photo. Rules compare amount, payee, date and reuse. This is advice and does not prove payment.

7. Approval. The owner or a service user confirms or rejects. The backend checks role, status, stock and receipt (transitionOrder()).

8. Result. The customer is told in chat, stock is updated and an event is logged. The dashboard updates live (LISTEN/NOTIFY).

Rules and AI. Rules do pricing, stock, receipt comparison and the approval gate, so the same input gives the same result. AI reads customer messages, calls tools, writes replies and reads receipt photos. Models: gpt-oss-120b and gpt-oss-20b on Groq (Llama and Qwen as fallback) and Gemini Flash. An adapter for the KrackedDevs hackathon gateway is first in line when its key is set.

People in control. Two roles, owner and service. Both can approve or reject; only the owner changes settings. The order stays pending until one of them acts. Rejecting needs a reason. Each action writes a row to the events table, and the Activity page reads those rows.

Security. Signed sessions, bcrypt password hashes, AES-256-GCM for stored channel credentials, workspace checks on every query (covered by tenant isolation tests), a Cloudflare Turnstile check and an emailed code at sign-up.

Orbyte fit. We looked at a demo Orbyte workspace and mapped Yapoz to what it shows. A pending order maps to Orbyte's Approve queue. Approved and rejected orders map to its approval states. A completed order maps to one charge on its credit wallet. Company Admin maps to our owner role and Approver to our

service role. src/lib/orbyte.ts builds four signed events: pending approval, approved, rejected and order completed. Each order card shows the events it has reached. Nothing is sent, because we have no access to Orbyte. The interfaces are assumptions, written up in docs/orbyte-integration.md.

Second business. A business is a workspace. Catalogue, hours, stock mode, payment accounts and policies are settings. We include a pizzeria (FORNO7), a hair salon (KASIH1) and a caterer (CATER1). The caterer was set up through the normal setup import with no new code. What it could not do is listed in docs/third-business-gaps.md.

3.2 Implementation & Hackathon Deliverables

Describe what was successfully built, integrated, and deployed during the 48-hour physical sprint. Highlight any specific problem-solving steps your team took to pivot or improve the project from the Phase 1 preliminary design.

[Maximum 300 words]

Since Phase 1 we moved the decision logic out of the interface and into the backend, and added features only where they serve the order journey.

Built and deployed:

- Approval gate. transitionOrder() runs in one locked database transaction. It refuses an order that is not pending, rechecks stock and receipts, and writes the activity event in the same transaction. A rejection needs a reason and returns the stock.
- Stock. Three modes: made to order, daily quantity and counted stock. A pending order holds its stock. With payments on, the hold lasts 20 minutes unless a receipt passes the checks.
- Receipts. Amount, payee, date and reuse checks, duplicate flags, and a way for staff to ask for another receipt.
- Appointments. Booking for service businesses, approved like orders.
- Activity page, limited by role.
- Orbyte. Four event payloads, HMAC signing, a preview on each order card and written interface assumptions.
- Privacy. Sign-up check and emailed code, a privacy notice on the first reply, and anonymising of finished orders.
- Measurement. benefit.ts works out the Phase 1 measures from system records.

Changes from the Phase 1 design:

- We removed Instagram and the hiring module (migration 0004). Hiring overlaps Orbyte's own Hiring module, and Instagram split our focus.
- Free AI tiers hit limits, so we added a router with fallbacks. If every provider fails, the customer gets a holding message and staff get an alert.

- We added tests for failures we hit, such as the AI failing mid-order and the AI naming dishes that are not on the menu.

The repository has 147 commits and over 250 automated tests.

4. Environmental & SDG Impact Verification

Validate the real-world impact of your built solution. Show concrete data that validate your environmental or social value proposition, compared against the baseline and KPI defined in previous phase submission.

*The data and evidence you provide in this section will be used to assess **SDG Impact, Technical Execution and Innovation scores.** Vague claims or missing verification will weaken your score across all three categories.*

Metric Category	**Your Response**
Target SDG Metric Achievement *Direct comparison between your Phase 1 baseline and your Phase 2 finalized prototype numbers* *e.g., "Achieved a 24.5% reduction in computational execution overhead against the 20% initial target."*	SDG 8, targets 8.2 and 8.3. Phase 1 baseline: 5 manual steps per order. Phase 2 result: 1.10 steps per order (target 1). That is 78% fewer than baseline against an 80% target. Not met. 20 of 20 orders arrived priced and stock-checked (target at least 90%). Met. 22 of 22 receipts were checked for amount, payee, date and reuse (target 100%). Met. The steps above 1 come from the two short-payment orders, which need a second receipt before approval. That is how the system is meant to work. All figures come from synthetic orders. In Phase 1 we said we would time the manual process on 20 orders. We did not, so we make no time-saved claim.
Verification Evidence Method *How was this data verified?*	npm run measure -- --synthetic places 20 orders through the real order, receipt and approval code in a demo workspace that it deletes afterwards. The mix is 16 clean

Metric Category	Your Response
e.g., codebase algorithmic profiling logs, hardware power-meter simulation readings, or database transaction counters.	orders, 2 short payments, 1 wrong payee and 1 reused receipt. src/lib/benefit.ts counts steps from the activity log, stock holds and receipt checks. The AI is left out of this run. We chose the mix, so it shows that the checks work, not how often real customers make these mistakes. Run on 31 Oct 2026 at commit e27e739. Full output and limits: docs/phase2-results.md. The owner's Today page shows the same measures from their own records.
Socio-Economic / Civic Value *How does your deployed application translate into a scalable, real-world solution capable of being supported by civic or corporate partners?*	MSMEs made up 39.7% of Malaysia's GDP and 48.7% of employment in 2025 (DOSM; includes medium firms). Our users are micro and small shops that run on chat. Fewer manual steps and fewer receipt mistakes give owners time back and cut lost or unpaid orders. No real owner has used Yapoz yet, so this value is not tested.
Deployment & Commercial Feasibility *e.g., modeled as an API service plugin designed for easy integration into existing corporate or institutional systems.*	Live at yapoz.vercel.app on Vercel and Supabase. Built as an order module for Orbyte: four documented events, one secret per business, every event tagged with its workspace. The Orbyte link is proposed, not live. Pricing: free tier, then a charge per completed order, paid from Orbyte's credit wallet. Not yet tested with owners.

4.1 Mathematical Proof / Code Logic Snippet (Optional, but holds bonus marks)

Paste the specific code section, mathematical formula, simulation log output, or calculation logic that proves the impact claims made above.

This proof is graded across three categories at once: ***Impact and Feasibility*** *(does the math support your claim),* ***Technical Execution*** *(does it reflect what you actually built), as well as* ***Innovation and Creativity*** *(is the approach original).] Do not leave this section generic or copied from Phase 1.*

The step count comes from what staff did before the decision, not from a number we typed in. From src/lib/benefit.ts:

```
export const BASELINE_STEPS = 5;
export const TARGET = { steps: 1, pricedShare: 0.9, receiptShare: 1 };

// steps = staff actions on the order up to the decision + staff chat messages
steps: decision
  ? mine.filter((a) => a.at <= decision.at).length + chats.length
  : null,

reduction: meanSteps === null ? null : 1 - meanSteps / BASELINE_STEPS
```

Working for the 20-order run: 18 orders took 1 step and 2 took 2 steps (the short payments).

```
mean steps = (18 x 1 + 2 x 2) / 20 = 22 / 20 = 1.10
reduction  = 1 - 1.10 / 5        = 0.78  (78%; target was 80%)
```

An order with no decision yet counts as no data, never as 0, and is left out of the mean.

5. Market Viability & Future Scalability Plan

SDC aims to encourage solutions that transition from innovations into market-ready applications. If given access to corporate funding, industry grants, or partnership opportunities (e.g., through SDC's Rotary network or joint research initiatives at XMUM), how would your team scale this prototype to process real-world data at volume?

[Maximum 250 words]

With funding, we would first test with real owners, then raise limits and make Yapoz easy for other builders to reuse.

1. Pilot (first two months). Put Yapoz in 5 to 10 real shops. Time owners on 20 orders by hand, then in Yapoz, to get a real time-saved figure. Rerun the measure script read-only on their records. Replace our assumed AI cost table with real invoices.

2. Production channels. Move WhatsApp from Meta's test number to an approved business number. Use paid AI plans so replies do not depend on free-tier limits.

3. Data and privacy. Move to a paid database plan close to Malaysia, set a retention period per business, and check PDPA points with a lawyer: who sees what, consent, deletion on request. We would not process real payments. Receipt checks stay advice.

4. Orbyte. Ask Orbyte the open questions in docs/orbyte-integration.md: how outside apps connect, how events are signed, which customer fields are allowed. Then build the sender with retries and a failed-delivery list. If Orbyte prefers to pull data, we expose a signed read-only route.

5. Pricing. Free tier, then a charge per completed order, not per seat, paid from Orbyte's credit wallet. We have not tested what owners would pay.

6. More businesses. Bakeries, salons, caterers and small retailers use the same flow with a different catalogue. We would add each new type through setup import and record what setup could not do.

6. Final Submission Checklist

Tick all boxes that apply before submitting.

- [X] The application is fully compiled, live, and accessible via the deployment link provided in Section 1.
- [X] The GitHub (or equivalent) repository contains a clean, well-documented README with installation instructions.
- [X] The GitHub (or equivalent) repository shows the history of incremental code updates made throughout the official live hacking window.
- [ ] The final pitch deck has been exported and saved as a flattened PDF file.
- [X] All fields in this template have been completed and word limits respected.

7. IP & Originality Declaration

7.1 Pre-Existing / Prior-Owned Work

Does this submission include any code, design, research, or other work that any team member created or owned before the start of this hackathon?

☐ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, briefly describe what was pre-existing and which team member(s) owned it:

7.2 Third-Party Code, Datasets, or Models

Does your project use any third-party code libraries, open-source frameworks, public/private datasets, or pre-trained AI models that your team did not create?

☒ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, list them below (open-source use is permitted provided it is declared and properly licensed/attributed):

Name	Type (code/dataset/model)	License
Next.js, React, React DOM, SWR, Zod, jose, clsx, date-fns, @date-fns/tz, qrcode, unpdf, pdf-lib, Tailwind CSS, tsx, ESLint, city-timezones, fflate, server-only	code (libraries)	MIT
Radix UI (dialog, dropdown, popover, switch, tabs, tooltip), Motion, Lenis	code (UI libraries)	MIT
Three.js, React Three Fiber, drei, postprocessing (landing page 3D)	code	MIT (postprocessing: Zlib)
GSAP and @gsap/react (landing page animation)	code	GSAP Standard "no charge" licence
Drizzle ORM, OpenAI SDK,	code	Apache-2.0

Anthropic SDK, TypeScript, Playwright		
postgres (postgres.js driver)	code	Unlicense
bcryptjs	code	BSD-3-Clause
grammY (Telegram bot framework), Drizzle Kit	code	MIT
web-push (browser push alerts)	code	MPL-2.0
Bricolage Grotesque, JetBrains Mono, Instrument Serif (Fontsource)	fonts	OFL-1.1
OpenAI gpt-oss-120b and gpt-oss-20b, served by Groq	pre-trained model (API)	Apache-2.0
Llama and Qwen models served by Groq (fallback only)	pre-trained model (API)	As published on Groq's model pages
Google Gemini Flash models (receipt reading, chat fallback)	pre-trained model (API)	Google Gemini API terms
Models behind the KrackedDevs hackathon gateway (used first when its key is set)	pre-trained model (API)	Not stated by the gateway
Cloudflare Turnstile (sign-up check), Resend (verification email), WhatsApp Cloud API (Meta), Telegram Bot API, Google Calendar API	external services (API)	Provider terms
Forno Sette, Kasih Hair Studio and Dapur Mak Long Catering (menus, chats, receipts)	dataset (synthetic)	Created by the team

7.3 Employment-Related IP Conflicts

Is any team member currently bound by an employment, internship, or consulting agreement under which their employer could claim ownership of work they create during this hackathon?

☐ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, please describe the nature of the agreement and which member it concerns:

8. Declaration

By submitting this report, the team leader confirms that the deployed application and data presented above are the original work of the team, completed within the official SDC 2026 hackathon period.

[Signature]

[Date]